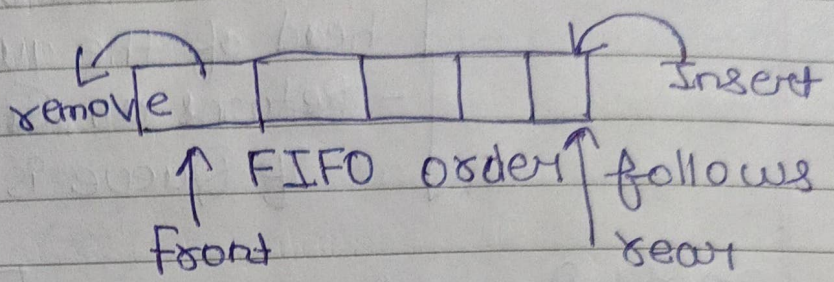


Methods are:

sort, binarySearch, get, reverse, shuffle, swap, copy, min, max, rotate, unmodifiableCollection.

Queue



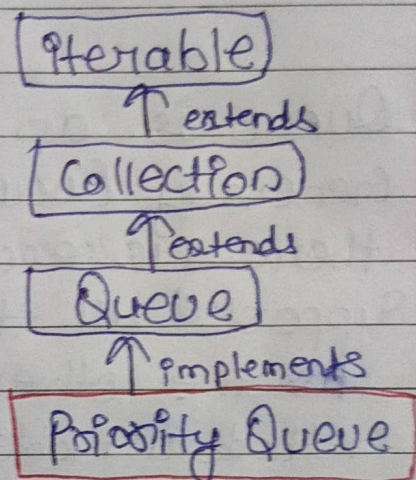
- Queue is an interface, child of Collection interface.
- Generally Queue follows FIFO approach, but there are exceptions like Priority Queue.
- Supports all the methods available in Collection + some other methods mentioned below.

Methods available

Sr.No	In Queue	Usage
1.	add()	- Insert the element into the queue - True if insertion is successful and Exception if insertion fails - Null element insertion is not allowed will throw (NPE)
2.	offer()	- Insert the element into the queue - True if insertion is successful and false if insertion fails - Null element insertion is not allowed will throw (NPE).
3.	poll()	- Retrieves and Removes the head of the queue - Returns null if Queue is Empty
4.	remove()	- Retrieves and remove head of Queue - Return Exception (NoSuchElementException) if Queue is Empty.

5. `peek()` - Retrieves the value present at the head of the queue but do not remove it.
- Returns null if Queue is Empty.
6. `element()` - Retrieves the value present at the head of the queue but do not remove it.
- Returns an Exception (`NoSuchElementException`) if Queue is Empty

`PriorityQueue` :



- Its of 2 types, Minimum `Priority Queue` and Maximum `Priority Queue`.
- It is based on priority Heap (Min / Max Heap)
- Elements are ordered according to either Natural Ordering (by default) or by Comparator provided during queue construction time.


```
public class Main {
```

```
    public static void main (String args[]) {
```

```
        // min priority queue, used to solve problems of
        // min heap.
```

```
        PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
        pq.add(5);
```

```
        pq.add(2);
```

```
        pq.add(8);
```

```
        pq.add(1);
```

```
        // lets print all the values
```

```
        while (!pq.isEmpty()) {
            int val = pq.poll();
            System.out.println(val);
        }
```

```
        // remove top element from PQ and print
```

```
        while (!pq.isEmpty()) {
```

```
            int val = pq.poll();
```

```
            System.out.println(val);
        }
```

```
    }
```

```
}
```

```
}
```

Output :

1

2

8

5

1

2

5

8

} removed elements

Ex: Shows how we declare
max priority Queue

```
PriorityQueue<Integer> maxpq =
new PriorityQueue<
(Integer a, Integer b) -> b-a>();
```

(Comparator)

Time complexity :

Add and offer : $O(\log n)$

peek() : $O(1)$

Poll and remove head element : $O(\log n)$

remove arbitrary element : $O(n)$

★★ Comparators Vs Comparable

→ Comparators and Comparable both provides a way to sort the collection of objects.

1. Primitive collection Sorting:

```
public class Main {
    public static void main() {
```

```
        int[] arr = {2, 1, 4, 3};
```

```
        Arrays.sort(arr); // Sorting in Ascending order
    } // Internally use DualPivotQuicksort
```

```
    }
    output: arr = [1, 2, 3, 4]
```

2. Object collection sorting:

```
public class Car {
```

```
    String carName;
```

```
    String carType;
```

```
    Car(String carName, String carType) {
```

```
        this.carName = carName;
```

```
        this.carType = carType;
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main() {
```

```
        Car[] cars = new Car[3];
```

```
        cars[0] = new Car("SUV", "Petrol");
```

```
        cars[1] = new Car("Sedan", "Diesel");
```

```
        cars[2] = new Car("HatchBack", "CNG");
```

```
        Arrays.sort(cars); // throw exception
```

```
    }
```

```
}
```


~~Error~~ Message: Exception in thread "main" java.lang.
ClassCastException: can cannot be cast to
java.lang.Comparable.

How to sort the Object Array?

with the help of

Comparator // functional interface

↓ contains

// abstract method

int compare(T obj1, T obj2);

Comparable

↗ interface

// has abstract method

int compareTo(T obj);

- Sorting algorithm uses this compare method of Comparator to compare 2 variables and decide whether to swap the variables or not.

Sorting algorithm uses this compareTo method of Comparator to compare variables and decide whether to swap the variable or not.

- Method returns;

1 : if $obj1 > obj2$ (Swap values)

0 : if $obj1 == obj2$ (No swap)

-1 : if $obj1 < obj2$ (No swap)

Mostly in algorithm, if this method return 1, it swap the value

Now sort canArray with the help of comparator

// sort based on canType

Arrays.sort(canArr, (can obj1, can obj2) →
 $obj1.canType.compareTo(obj2.canType)$);

// sort based on canName

Arrays.sort(canArr, (can obj1, can obj2) →
 $obj1.canName.compareTo(obj2.canName)$);

↑
comparision with compareTo

If we need to sort Car Array only in asc/des order already known then we can define sorting order of object in the class declaration time.

```

ex: public class Car implements Comparator<Car> {
    String carName;
    String carType;
    Car() {}
    Car(String cName, String cType) {
        this.carName = cName;
        this.carType = cType;
    }
    @Override
    public int compare(Car ob1, Car ob2) {
        return ob1.carName.compareTo
            (ob2.carName);
    }
}

public class Main {
    psum() {
        List<Car> cars = new ArrayList<>();
        cars.add(new Car("SUV", "Petrol"));
        cars.add(new Car("Sedan", "Diesel"));
        cars.add(new Car("Hatchback", "CNG"));

        Collections.sort(cars, new Car());
        cars.forEach((Car car) -> out.println(car.carName));
    }
}

```

Output: Hatchback
Sedan
SUV

11 Sort Array of object using comparable based on carName.

```
public class Car implements Comparable<Car> {
    String carName;
    String carType;
    Car(String carName, String carType) {
        this.carName = carName;
        this.carType = carType;
    }
    @Override
    public int compareTo(Car obj) {
        return this.carName.compareTo(obj.carName);
    }
}
```

```
public class Main {
    public static void main() {
        List<Car> cars = new ArrayList<>();
        cars.add(new Car("SUV", "petrol"));
        cars.add(new Car("Sedan", "diesel"));
        cars.add(new Car("Hatchback", "CNG"));

        Collections.sort(cars);
        cars.forEach((Car car) -> System.out.println(car.carName));
    }
}
```

Output: Hatchback
Sedan
SUV

Disadvantage of Comparable is only one type of sorting order possible but with comparable we can define multiple sorting order of same collection.